SYSTEM DESIGN

Design Netflix

Video streaming platform serving 250M+ subscribers, 15M concurrent streams globally

~ 28 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Statement & Clarifying Questions
02 · Functional & Non-Functional Requirements
03 · Capacity Estimation
04 · High-Level Architecture
05 · Open Connect CDN Deep Dive
06 · Video Encoding Pipeline
07 · Adaptive Streaming (DASH)
08 · Microservices & Resilience
09 · Database Architecture
10 · Recommendation System
11 · Data Pipeline
12 · Resilience & Chaos Engineering
13 · Failure Modes & Recovery
14 · Trade-offs
15 · Interview Playbook

01 Problem Statement & Clarifying Questions

Define the scope of the Netflix system we're designing.

Scope	Question	Assumption
Video Types	Streaming only, or include downloads?	Streaming primary; downloads secondary
Live Content	Live TV / sports, or VOD only?	Primarily VOD; live is edge case
DRM	Content protection required?	Yes — Widevine, PlayReady, FairPlay
Profiles	Support multiple user profiles per account?	Yes — child, adult, recommendations per profile
Geographic	Global deployment required?	Yes — 190+ countries, localized CDN
Recommendations	Personalized homepage essential?	Yes — Stage 1,2,3 ranking pipeline
Offline	Offline viewing (downloads)?	Yes but secondary focus

We'll focus on the streaming architecture for VOD content globally, with DRM, personalized recommendations, and multi-profile support.

02 Functional & Non-Functional Requirements

Functional Requirements

- Browse/search catalog (metadata API): instant (<100ms)
- Get personalized recommendations: list 100 tiles in <200ms
- Initiate playback: fetch OCA URL, DRM token within <500ms
- Stream video continuously: adaptive bitrate, seamless quality switches
- Track viewing history: record play/pause/seek/stop events
- Multi-profile management: switch profiles instantly
- Download for offline: queue & background sync

Non-Functional Requirements

- **Availability:** 99.95% uptime SLA; graceful degradation if recommendation/billing service fails
- **Latency:** Initial playback <3s; segment delivery <50ms p99; UI response <100ms
- **Scale:** 250M subscribers; 15M concurrent streams; 5K+ titles; 700B events/day

© Study Guide — For personal use only

- **Quality:** Adaptive bitrate (240p–4K); device-specific codecs; audio/subs multi-lang
- **Security:** End-to-end DRM; HTTPS; rate limiting; account security
- **Cost:** Minimize CDN egress; aggressive caching; strategic content placement

03 Capacity Estimation

User & Concurrency

- **250M subscribers:** active base; 30–40% watch on any given day (~100M daily active)
- **Peak hour:** 15M concurrent streams (typical: 8–10M; peak: ~15M in US/EU primetime)
- **Device mix:** 40% TV, 45% mobile, 15% web/tablet
- **Avg watch session:** 1–2 hours; avg bitrate: 5Mbps (range 1–25Mbps per device profile)

Bandwidth & CDN

- **Peak CDN traffic:** 15M streams × 5Mbps = 75 Tbps
- **Annual content egress:** ~350–400 EB (exabytes) from origin/caches to users
- **OCA hit rate:** 99%+ of requests served from ISP-embedded appliances; <1% miss → regional PoP or origin
- **Cache efficiency:** Proactive nightly push of top-N content; reactive caching for long-tail

Storage

- **Catalog size:** ~5K titles; avg 100GB per title (all resolution/codec variants) = 500TB aggregate
- **Viewing history:** ~500B viewing history records (user_id, title_id, position, timestamp); ~100GB hot store in Cassandra
- **Metadata:** ~100GB (posters, descriptions, reviews, ratings, availability matrix)
- **Event logs:** 700B events/day × 7 days retention = ~2PB in Kafka/cloud storage

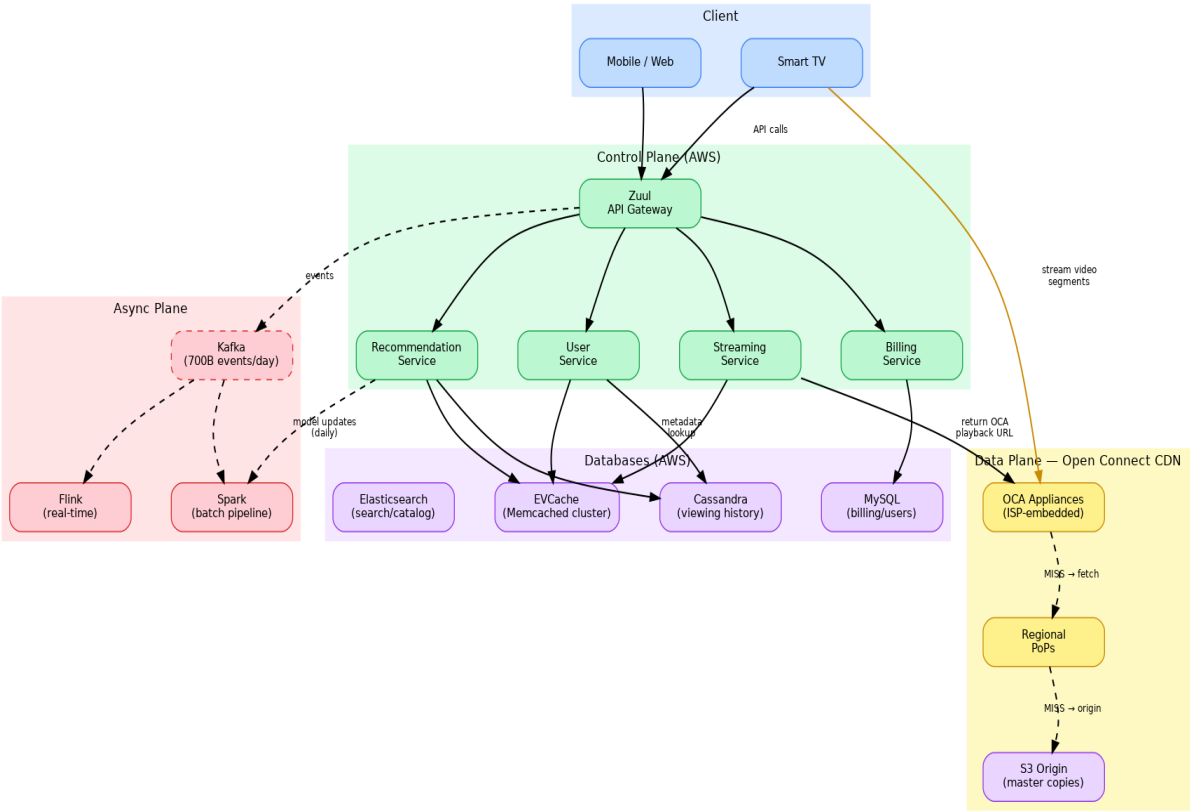
Cache Layer

- **EVCache (Memcached):** ~800M queries/day for metadata, viewing history, OCA URL lookup; target 99% hit rate; ~10TB heap across cluster
- **Elasticsearch:** ~2TB for searchable catalog + user ratings/reviews; shard by region

04 High-Level Architecture

Netflix separates the system into three distinct planes:

 © Study Guide — For personal use only



Control Plane (AWS)

- **Zuul API Gateway:** Request routing, authentication, rate limiting; filters for logging, metrics, DRM token generation
- **User Service:** Account management, profiles, settings; backed by MySQL (ACID for billing)
- **Streaming Service:** Core orchestrator; determines OCA endpoint for client IP; queries EVCache for metadata/URL mappings
- **Recommendation Service:** Three-stage ranking engine; candidate generation → filtering → ranking (150+ features)
- **Billing Service:** Subscription state, payment processing, entitlements; MySQL + event journal

Data Plane (Open Connect CDN)

- **OCA Appliances:** Custom hardware (100TB SSD, FreeBSD OS) embedded in 1,000+ ISPs globally; serves ~99%+ of video segments; ultra-low latency (~5ms)
- **Regional PoPs:** Fallback caches in major cloud regions (AWS regions, GCP locations); serve cache misses from OCA
- **S3 Origin:** Master video copies; proactive nightly push to OCA appliances via BGP-controlled content distribution

Async Plane (Event-Driven)

- **Kafka:** Ingest 700B events/day (play, pause, seek, rate, search, errors); distributed topics by event type; 7-day retention

 © Study Guide — For personal use only

- **Spark:** Daily batch jobs; model retraining for recommendations; offline analytics; data warehouse ETL
- **Flink:** Real-time stream processing; trending content; live metrics dashboards; anomaly detection

05 Open Connect CDN Deep Dive

Why Netflix Doesn't Use AWS CloudFront

Netflix's proprietary CDN, Open Connect (OCA), is the backbone of the data plane. Cost and performance are the drivers.

Why Custom CDN?

- **Cost:** CloudFront egress ~$0.085/GB; Netflix's OCA: $0.01–$0.02/GB (5–10x cheaper due to ISP partnerships, shared infrastructure, bulk agreements)
- **Control:** Content placement, cache eviction, codec/resolution variants per ISP; no third-party provider constraints
- **Scale:** 30–40% of global internet traffic is Netflix; no CDN can compete on efficiency
- **Latency:** ISP-embedded appliances serve from end-users' local networks; 5ms latency vs. 20–50ms from regional edge

OCA Hardware & Deployment

- **Appliance specs:** Custom FreeBSD appliance; 100TB SSD; 10Gbps NIC; low power, silent operation for ISP NOCs
- **Placement:** 1,000+ ISPs across 190+ countries; serves ~50–60% of peak global traffic
- **Regional PoPs:** AWS/GCP regions + backbone providers as fallback for OCA misses

Content Distribution Strategy

- **Proactive caching (nightly push):** Netflix predicts top-N content for each ISP based on regional popularity, time zones, device profiles; BGP-announced routes push content during off-peak hours
- **Reactive caching:** OCA misses → fetch from regional PoP or S3; cache result for future requests (LRU eviction)
- **Cache hit rate:** Target 99%+ for top-50% of catalog; 90%+ for long-tail
- **Proactive refresh:** Pre-position new releases, trending content; ISP appliances auto-sync nightly

Segment & Manifest Delivery

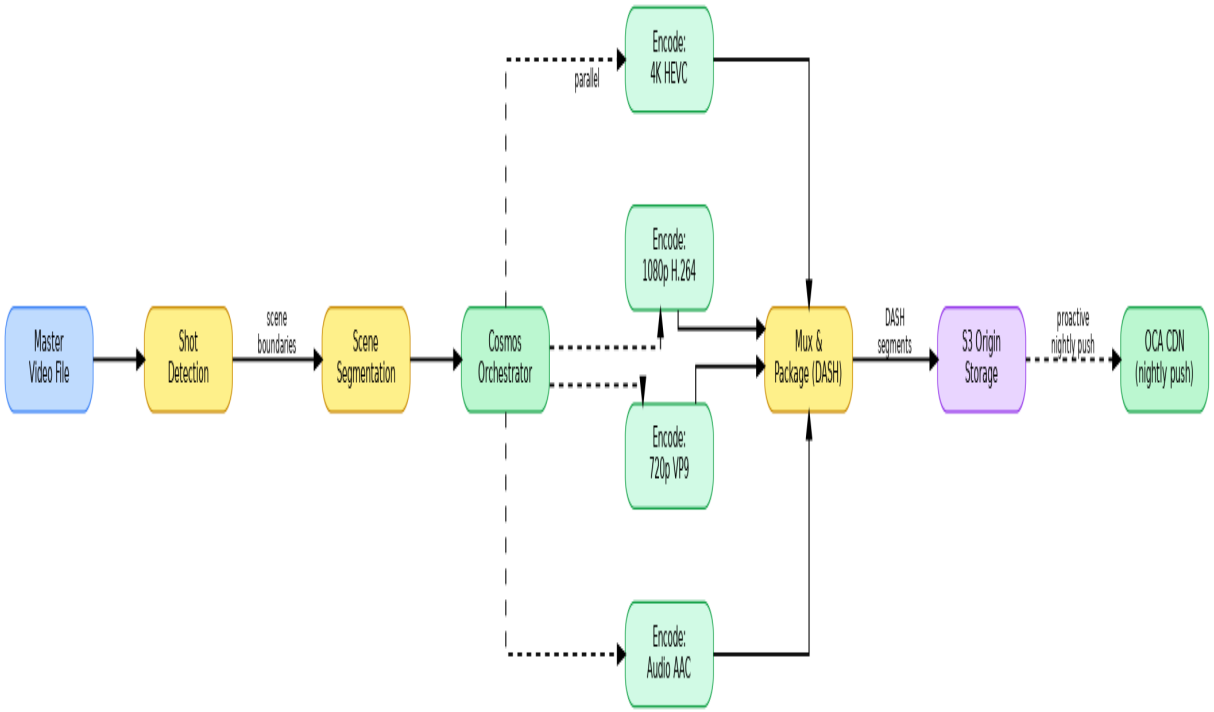
- **Manifests:** Device-specific DASH manifests returned from Streaming Service (metadata service); includes optimal OCA URL for client's ISP
- **Segments:** Client fetches 2–4 second MPEG-DASH segments directly from OCA URL; adaptive bitrate per segment based on connection speed
- **Fallback logic:** If OCA unavailable (network issue), client retries regional PoP then origin

© Study Guide — For personal use only

06 Video Encoding Pipeline

Cosmos & Archer Orchestration

Netflix encodes every title into 100+ variants (codec/resolution/bitrate combinations) to optimize for device capabilities and network conditions.



Per-Title Encoding (Revolutionary)

- **Shot detection:** Analyze master file for shot boundaries, scene complexity, motion intensity
- **Variable bitrate by scene:** Complex scenes (explosions, CGI) get higher bitrate; simple scenes (dialogue, static) get lower bitrate
- **Result:** 20–30% bandwidth savings vs. uniform bitrate at same perceptual quality
- **Example:** 1080p H.264: 5Mbps average, but 8Mbps for action scenes, 2Mbps for dialogue

Codec Ladder & Device Profiles

Device Profile	Codec Preference	Resolutions	Bitrate Range
Smart TV (2023+)	HEVC (H.265), VP9	4K, 1080p	8–25 Mbps
Modern Smartphone	H.264, VP9	1080p, 720p	3–8 Mbps
Older Mobile/Tablet	H.264 only	720p, 480p	1–4 Mbps
Web Browser	H.264, VP9	1080p, 720p	3–8 Mbps

© Study Guide — For personal use only

Device Profile	Codec Preference	Resolutions	Bitrate Range
Fire Stick	H.264, HEVC	4K, 1080p	5–15 Mbps

Cosmos Orchestration

- **Cosmos:** Distributed encoding orchestrator; schedules parallel transcoding jobs across Netflix's encoding fleet
- **Archer:** Encoding engine; runs FFmpeg (with custom plugins for per-title bitrate optimization) on CPU/GPU clusters
- **Parallel encoding:** Each title encoded to all variants in parallel (4K HEVC, 1080p H.264, 720p VP9, audio AAC, etc.)
- **Cost:** ~$0.25–$0.50 per hour of video encoded; 5K titles × 100 variants = 500K encoding jobs; ~$2–5M annual encoding cost

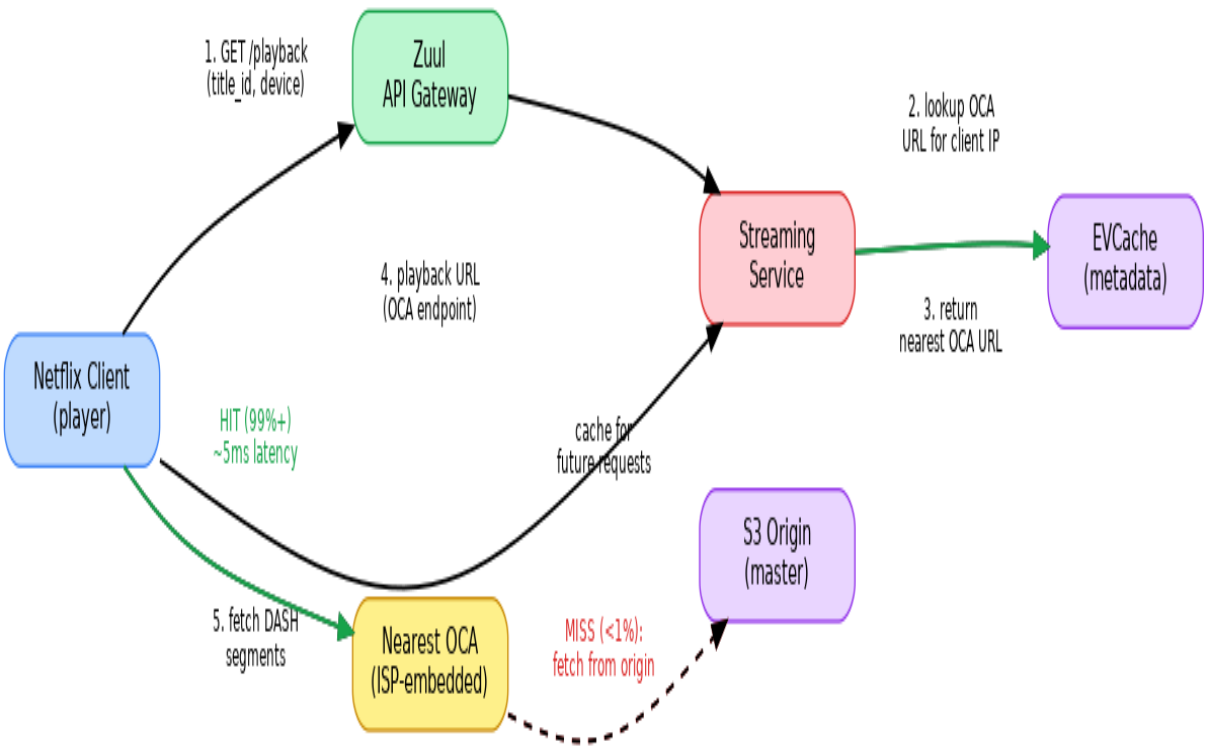
Packaging & Distribution

- **DASH (Dynamic Adaptive Streaming over HTTP):** Industry-standard format; client-side adaptive bitrate selection
- **Segmentation:** 2–4 second segments; manifest lists all available qualities; player chooses based on bandwidth
- **DRM packaging:** Widevine, PlayReady, FairPlay licenses embedded per variant
- **S3 storage:** Master copies; nightly batch push to OCA appliances via BGP-directed distribution

07 Adaptive Streaming (DASH)

Client-Side Quality Selection

© Study Guide — For personal use only



Playback Flow

1. Client calls Zuul /playback API with title_id, device profile, user IP
2. Streaming Service queries EVCache for OCA URL mapping (geo-aware; returns nearest ISP appliance)
3. EVCache returns optimal OCA endpoint for client's IP range
4. Zuul returns playback URL + DASH manifest to client
5. Client fetches DASH manifest from manifest server (metadata)
6. Client initiates segment fetch from OCA endpoint
7. OCA serves segment from local cache (HIT ~99%+) or fetches from regional PoP (MISS <1%)

BOLA Algorithm (Buffer Occupancy-Based Bitrate Adaptation)

- **Goal:** Maximize bitrate while avoiding stalls; minimize re-buffering events
- **Key metric:** Buffer occupancy (seconds of video buffered locally); not network bandwidth alone
- **Logic:** If buffer >30s, increase bitrate quality; if buffer <10s, decrease quality; target ~15s buffer
- **Advantage:** More stable quality switching than network-speed only; reduces stalls in congested networks
- **SLA:** Stall rate <0.5% (less than 3 seconds stalled per hour of viewing)

Segment Details

- **Segment duration:** 2–4 seconds; granular quality switching; ~50KB–5MB per segment depending on quality
- **Prefetching:** Client prefetches 3–5 segments ahead to maintain buffer

 © Study Guide — For personal use only

- **Playback URL includes geo-aware ISP detection:** IPv4 GeoIP lookup determines nearest OCA appliance
- **Fallback chain:** OCA → Regional PoP → S3 origin; client retries with exponential backoff

08 Microservices & Resilience

How Netflix Avoids Catastrophic Failures

Stateless Services

- **Zuul, Streaming Service, Recommendation Service:** All stateless; horizontally scalable; request routed to any instance
- **No local caches or sessions:** All state in EVCache or databases; failures don't lose in-flight state
- **Load balancing:** Eureka (service discovery) + Ribbon (client-side LB) for intelligent request routing

Hystrix Circuit Breaker (Resilience Pattern)

- **Fallback catalog:** If Recommendation Service fails, fall back to 'Popular Now' or 'Trending Globally' (pre-computed offline)
- **Circuit breaker logic:** Fail fast; don't retry downstream if service is down; return cached response or 'degraded' UI
- **Metrics:** Error rate, latency p99, timeouts trigger circuit open; halfOpen state allows slow recovery testing
- **Timeouts:** All inter-service calls have <500ms timeout; fail fast rather than cascade latency

Chaos Engineering

- **Chaos Monkey:** Randomly kills instances during business hours; team must architect for recovery
- **Chaos Kong:** Kills entire AWS region; tests multi-region failover (Route53 latency-based routing)
- **Latency Monkey:** Injects artificial delays to test timeout handling and fallback logic
- **Culture:** Failure injection is proactive testing; failures found in chaos tests don't surprise production

Service Discovery (Eureka) & Client-Side Load Balancing (Ribbon)

- **Eureka:** Central service registry; services register heartbeat every 30s; Netflix internal DNS alternative
- **Ribbon:** Client-side LB; caches service instance list from Eureka; retries failed requests to healthy instances
- **Benefit:** No single LB bottleneck; intelligent request routing considers instance health, latency zones, canary deployments

09 Database Architecture

MySQL + Cassandra + EVCache + Elasticsearch

© Study Guide — For personal use only

MySQL (Billing, Users, ACID)

- **Use case:** User accounts, subscription tier, billing history, payment methods, entitlements
- **Why MySQL:** ACID guarantees essential; billing must be precise; transactions required for payment processing
- **Scale:** ~250M user records; write-heavy (daily billing updates, subscription changes); replicated across multi-master setup
- **Sharding:** By user_id modulo; thousands of shards to distribute load

Cassandra (Viewing History, Write-Heavy)

- **Use case:** Viewing history (play position, watched status, resume timestamp); ~500B records; append-only log
- **Why Cassandra:** Extremely write-heavy; distributed design; no single point of failure; tunable consistency (eventual ok for history)
- **Data model:** Wide rows by user_id; columns = {title_id: {position, timestamp, watched}}
- **Throughput:** ~2.5K ops/sec per node; >100 nodes in cluster; millions of writes/sec aggregate
- **Consistency:** Eventual consistency (RF=3, quorum reads/writes for critical data)

EVCache (Memcached, Metadata & Playback URLs)

- **Use case:** Metadata (title, poster, description), OCA playback URLs, viewing history snippets
- **Scale:** ~800M hits/day; 99% hit rate target; <30ms p99 latency
- **Cluster:** ~10TB total heap; distributed across 100s of nodes; replicated 2–3x
- **Eviction:** LRU when memory full; keys expire after 24h (auto-refresh from source)
- **Key benefit:** Avoid expensive Cassandra or S3 lookups; metadata lookups now in-memory

Elasticsearch (Search & Catalog)

- **Use case:** Full-text search (title name, description, actor, director); faceted search (genre, year, rating)
- **Scale:** ~2TB index across 50+ shards; shard by region/language for locality
- **Consistency:** Near real-time; new titles indexed within minutes
- **Features:** Typo tolerance, synonym matching, personalized results (user rating history influences ranking)

10 Recommendation System

Three-Stage Ranking Pipeline

Netflix's homepage personalizes ~100 rows per user, each with 10–30 titles. The algorithm runs in three stages to balance quality and speed.

Stage 1: Candidate Generation (~10K candidates per user)

 © Study Guide — For personal use only

- **Algorithm:** Alternating Least Squares (ALS) matrix factorization; collaborative filtering
- **Input:** User-title interaction matrix (viewing history, ratings, search, clicks); item embeddings (metadata features)
- **Output:** Top 10K candidate titles per user (not yet ranked; sorted by relevance score)
- **Scale:** Batch job; retrains daily using Spark; maps users and titles to low-dimensional vectors
- **Latency:** Generation is offline; lookup at runtime is O(1) hash or in-memory vector search

Stage 2: Filtering (~1–2K candidates after filtering)

- **Rules:** Remove already-watched titles, region-locked content, age-restricted titles not allowed for user profile
- **Business logic:** Prioritize licensed content expiring soon; boost originals; remove low-quality titles
- **Latency:** <100ms; deterministic rules; can be computed in real-time or cached

Stage 3: Ranking (~100 titles to display; per-row ranking)

- **Algorithm:** Gradient Boosting (LightGBM); predict CTR (click-through rate) for title in context
- **Features (150+):** User features (age, country, subscription tier), item features (genre, IMDb score, release date), context (time-of-day, device, row position)
- **Training:** Offline daily batch; Spark pipeline on Kafka events (clicks, watches, searches)
- **Personalization:** Artwork selection per user (e.g., show thumbnail of hero actor user has rated high; or action scene if user watches action movies)
- **A/B testing:** Homepage layout, row ordering, thumbnail variants all A/B tested; winner picked based on CTR, watch time

Why Three Stages?

- **Speed:** Each stage reduces candidate set; Stage 3 ranking only runs on 2K titles, not 5K+
- **Quality:** Specialized algorithms at each stage; ALS captures broad taste; LightGBM captures interaction effects
- **Cost:** ALS + filtering offline; only LightGBM runs at request time; <200ms total latency
- **Explainability:** Can inspect why title ranked high (user watched similar title; high score; popular now)

11 Data Pipeline

Kafka → Spark/Flink → Analytics

Kafka Ingest (700B Events/Day)

- **Event types:** play, pause, seek, stop, rate, search, error, subscription change, profile switch
- **Volume:** 250M users × 2–3 events per minute (during watching) = 700B events/day
- **Schema:** JSON; timestamp, user_id, title_id, device, playback position, event type, client IP
- **Retention:** 7 days hot; older events archived to S3 for long-term analytics

© Study Guide — For personal use only

- **Partitioning:** By user_id or event_type; ensures ordering per user

Spark Batch Pipeline (Daily)

- **Jobs:** Model retraining (ALS, LightGBM), data warehouse ETL, offline analytics, reporting
- **Input:** Kafka events + viewing history (Cassandra snapshots) + content metadata
- **Output:** Trained models (ALS embeddings, LightGBM weights) → deployed to online serving; aggregated metrics → Druid
- **Latency:** ~4 hours per daily job; models deployed by morning for next day's recommendations

Flink Real-Time Stream (Live Metrics)

- **Jobs:** Trending content (titles gaining play rate in last hour), anomaly detection (error spikes, stalls), live dashboards
- **Latency:** <5 second latency; results published to Kafka topics or websockets for dashboards
- **Stateful operations:** Count plays per title per minute; detect >3σ deviations as anomalies

Druid OLAP (Ad-Hoc Analytics)

- **Use case:** Dashboard queries (titles watched per country, device, subscription tier); near real-time data exploration
- **Metrics:** Plays, stall rate, bitrate distribution, recommendations CTR
- **Latency:** <1s query latency for pre-aggregated dashboards; <5s for ad-hoc analysis

12 Resilience & Chaos Engineering

How Netflix Survives Disasters

Multi-Region Active-Active Deployment

- **Regions:** US-East, US-West, EU-West, APAC (minimum 3 active regions)
- **Traffic routing:** Route53 latency-based routing; routes user to nearest region
- **Database replication:** MySQL multi-master (cross-region); Cassandra multi-DC (RF=3 across datacenters)
- **Failover:** If region fails, Route53 automatically reroutes traffic; no manual intervention
- **Data loss:** RPO <5 minutes (replication lag); RTO <30 seconds (Route53 + load balancer failover)

Chaos Monkey (Instance-Level Failures)

- **Action:** Randomly terminates instances during business hours (not in production automatically, but in staging/pre-prod)
- **Frequency:** One instance per service per region per hour
- **Culture:** Team must be on-call; assume instances will fail; architect accordingly

© Study Guide — For personal use only

- **Benefit:** Discovers single points of failure before production incidents

Chaos Kong (Region-Level Failures)

- **Action:** Simulates entire AWS region failure; yanks network cables (test mode in staging)
- **Test:** Verify Route53 failover, database failover, client reconnection logic all work
- **Cadence:** Quarterly; part of disaster recovery drill

Latency Monkey (Network Delays)

- **Action:** Injects 100–500ms artificial delay on inter-service calls
- **Purpose:** Ensure all timeouts are configured correctly; fallback logic triggers before timeout
- **Result:** Prevents cascade failures when one service degrades; circuit breaker opens gracefully

13 Failure Modes & Recovery

8 Critical Scenarios

Failure	Cause	Impact	Mitigation
OCA Appliance Outage	Hardware failure; ISP power loss	Users in that ISP routed to regional PoP; latency increases 5ms → 50ms; 1–5% QoE impact	Proactive health checks; automatic fallback chain; redundant appliances per ISP
Cassandra Partition	Network split; DC unreachable	Viewing history writes fail; reads serve stale data (eventual consistency)	Quorum reads/writes; RF=3 across regions; application fallback to cache
EVCache Miss Storm	Memcached cluster crash; cache stampede	Metadata lookups hit Cassandra; p99 latency spikes to 500ms	Circuit breaker; fallback to cached manifest; preload hot keys at startup
Recommendation Lag	Model training delayed; batch job failure	Homepage shows generic 'Popular Now' instead of personalized rows	Serve previous day's model; A/B test impact minimal (CTR diff <1%)
Encoding Pipeline Failure	Cosmos orchestrator down; encoder crash	New title cannot be encoded; delays release	Manual restart; fallback to on-demand encoding; pre-compute common profiles
DRM Service Down	License server unavailable	Playback blocked; users cannot watch any content	Cache DRM licenses locally for 24h; fallback to offline viewing if downloaded
Zuul Gateway Overload	Thundering herd; DDoS attack	API latency >1s; some requests timeout and retry	Rate limiting; circuit breaker; shed load by returning 429 (backoff client)

© Study Guide — For personal use only

Failure	Cause	Impact	Mitigation
Billing Service Failure	MySQL replication lag; payment processor down	Subscription state unknown; allow stream to continue (billing retry later)	Optimistic UI; async reconciliation; manual dispute resolution

14 Trade-offs & Design Decisions

Why These Choices?

Decision	Alternative	Why Netflix Chose It
Own CDN (OCA)	AWS CloudFront / Akamai	Cost (5–10x cheaper); control over content placement; latency (ISP embedding)
Cassandra	DynamoDB / MongoDB	Write throughput (2.5K ops/sec per node); no SPOF; operational maturity
DASH (HTTP)	RTMP / custom protocol	Firewall-friendly; standards-based; adaptive bitrate client-controlled
Eventual Consistency	Strong consistency	Viewing history 'eventual ok'; availability & latency >> precision
Separate 3 Planes	Monolith	Independent scaling; failure isolation; team ownership
Three-Stage Ranking	Single model	Latency <200ms achieved by filtering + LightGBM only on 2K candidates
Daily Model Retraining	Real-time updates	Cost/complexity trade-off; daily refresh sufficient for CTR; A/B tests catch bugs
Proactive CDN Push	Reactive only	Bandwidth savings; 99%+ hit rate; scale advantages at Netflix's size

15 Interview Playbook

How to Present This Design (45 Minutes)

Opening (5 min): Clarifying Questions & Scope

- Start with 2–3 clarifying questions: VOD only? Global? Recommendations essential?
- Confirm assumptions: 250M subscribers, 15M concurrent streams, 5K titles, multi-region
- Set scope: Focus on streaming architecture (not downloads or live)

Problem (2 min): Capacity Estimation

© Study Guide — For personal use only

- **Memorize:** 250M subscribers, 15M concurrent (peak), 75 Tbps CDN traffic, 500TB catalog
- Draw on whiteboard: User base → concurrent → bandwidth calculation
- Point out the scale problem: Standard CDN (AWS CloudFront) costs prohibitive

Architecture (10 min): 3 Planes + Diagrams

- **Control plane:** Zuul → microservices (User, Billing, Streaming, Recommendation); stateless + Hystrix fallback
- **Data plane:** Open Connect CDN; 1,000+ ISP-embedded appliances; 99%+ hit rate; nightly proactive push
- **Async plane:** Kafka (700B events/day) → Spark (daily retraining) → Flink (live metrics)
- Use diagrams to illustrate data flow; explain why custom CDN is critical for cost & scale

Deep Dives (20 min): Pick 2–3 Topics

- **Option 1 (CDN Focus):** Explain OCA hardware, ISP partnerships, proactive vs. reactive caching; why 99%+ hit rate
- **Option 2 (Recommendation):** Three-stage ranking; ALS generation → filtering → LightGBM ranking; 150+ features; personalization
- **Option 3 (Encoding):** Per-title encoding; scene-based variable bitrate; codec ladder by device; Cosmos orchestration
- **Option 4 (Resilience):** Chaos Monkey/Kong; Hystrix circuit breaker; multi-region active-active; failure mode table
- Interviewers often probe: 'How would you handle Cassandra partition?', 'Why not use CloudFront?', 'How do you scale recommendations?'

Wrap-Up (5 min): Trade-offs & Tradeoffs

- Discuss cost vs. quality vs. latency tradeoffs
- Mention operational complexity (Chaos Monkey culture, multi-region databases)
- End with: 'Netflix is unique at scale; most companies don't justify custom CDN, but Netflix streams 30–40% of global internet traffic'

Key Numbers to Memorize (Quiz Yourself)

- 250M subscribers, 15M concurrent streams, 75 Tbps CDN traffic
- OCA hit rate: 99%+; latency: 5ms vs. 50ms regional
- 700B events/day to Kafka; daily model retraining
- Cassandra: 2.5K ops/sec per node; EVCache: 99% hit rate; 800M queries/day
- DASH 2–4s segments; BOLA buffer algorithm; <0.5% stall rate SLA
- Encoding: 100+ variants per title; per-title VBR saves 20–30% bandwidth

What Interviewers Are Testing

 © Study Guide — For personal use only

• **Systems thinking:** Can you connect pieces? (Why custom CDN affects encoding, caching strategy, etc.)

• **Scale intuition:** 15M concurrent = how much bandwidth? How many database operations?

• **Trade-off reasoning:** Why OCA over CloudFront? Why Cassandra over MySQL?

• **Resilience:** What fails? How do you recover? Do you have a fallback?

• **Communication:** Can you explain complex system simply? Use diagrams. Admit uncertainty.

 © Study Guide — For personal use only

Additional Resources

- **Netflix Tech Blog:** 'Rethinking Streaming Media Architectures' (recommended reading for deep dives)
- **Open Connect White Papers:** Netflix publishes detailed OCA technical reports
- **YouTube Talks:** 'Netflix Chaos Engineering' (Chaos Monkey & Kong talks)
- **System Design Interview Prep:** Practice explaining each plane; draw diagrams repeatedly until you can explain without looking at notes

© Study Guide — For personal use only